Program Studi Teknik Informatika
Fakultas Teknologi Industri, Institut Teknologi Sumatera

IF25-11002 PRAKTIKUM PEMROGRAMAN

Modul Belajar 9

Array Satu Dimensi

Menyimpan banyak nilai sejenis, mengaksesnya dengan indeks, dan memprosesnya dengan perulangan

Tim Pengajar
Pendamping Pertemuan 9. Tahun 2026

Komponen	Keterangan
Pertemuan	9 dari 16
Capaian	CPMK0607 (menerapkan) dan CPMK0608 (menjelaskan) pada konsep dasar algoritma dan sistem komputer
Waktu belajar	sekitar 3 jam belajar mandiri, ditambah 150 menit di kelas
Hands-on	`02 Hands-on/P9 - Array 1D - Hands-on/`

MODUL 9

Peta belajar

Ikuti urutan ini dari atas ke bawah. Setiap bagian memakai apa yang sudah dibahas di bagian sebelumnya.

Urutan	Bagian
1	Tentang modul ini
2	Masalah pembuka: Siapa yang di atas rata-rata?
3	Langkah 1: Deklarasi dan indeks
4	Langkah 2: Mengisi array dari masukan
5	Langkah 3: Mencari maksimum dan posisinya
6	Langkah 4: Dua putaran: rata-rata lalu bandingkan
7	Langkah 5: Di luar batas
8	Langkah 6: Menggeser dan membalik
9	Pesan error yang sering muncul
10	Latihan mandiri
11	Rangkuman
12	Cek pemahaman
13	Exit Ticket dan tugas
14	Bacaan lanjut dan persiapan minggu depan

PEMBUKA

Tentang modul ini

Modul ini mendampingi Pertemuan 9, awal paruh kedua semester. Sampai UTS, setiap variabel menyimpan satu nilai. Minggu ini kalian memakai array untuk menyimpan banyak nilai sejenis di bawah satu nama, sehingga data bisa diproses berulang kali.

Isinya sama dengan slide di kelas, tetapi ditulis lebih pelan dan lebih lengkap, supaya kalian bisa mengulang sendiri di rumah tanpa harus menebak maksud slide.

Setelah menyelesaikan modul ini, kalian bisa

No	Kemampuan	Dibahas di
1	Menulis program yang menyimpan data sejenis di array satu dimensi dan memprosesnya dengan perulangan: mengisi, menampilkan, menjumlahkan, mencari maksimum, dan menggeser (Sub-CPMK 9.1, CPMK0607)	Langkah 1 sampai 6
2	Menelusuri isi array sesudah setiap langkah pemrosesan dan menjelaskan kesalahan indeks, termasuk akses di luar batas (Sub-CPMK 9.2, CPMK0608)	kotak Tebak dulu, Latihan 3

Karena setiap instrumen penilaian dibagi rata antara Bagian A (menulis kode) dan Bagian B (menelusuri dan menjelaskan), yang dinilai bukan hanya program kalian jalan, tetapi juga kalian bisa menjelaskan mengapa program itu jalan.

Cara memakai modul ini

Setiap langkah punya pola yang sama: penjelasan konsep, kadang contoh dari kehidupan sehari-hari, lalu kode yang bisa langsung kalian ketik. Sesudah kode ada kotak **Tebak dulu**. Tulis tebakan kalian di kertas sebelum membaca **Jawaban** di bawahnya. Kebiasaan menebak lalu membuktikan inilah yang membuat konsep menempel, bukan sekadar membaca.

> PENTING **Ketik, jangan salin**
>
> Ketik ulang setiap contoh di GDB Online atau editor kalian, jangan disalin-tempel. Saat mengetik, kalian akan membuat salah ketik, membaca pesan error, dan memperbaikinya. Itu bagian dari belajar.

Yang perlu disiapkan

Peramban dengan akses ke onlinegdb.com, atau g++ di komputer kalian sendiri. Kertas dan alat tulis untuk menebak keluaran dan membuat trace table. Folder hands-on pertemuan ini dari repo mata kuliah.

PEMBUKA

Masalah pembuka: Siapa yang di atas rata-rata?

Dosen ingin tahu berapa mahasiswa yang nilainya di atas rata-rata kelas.

Dengan pola sentinel minggu 5, rata-rata baru diketahui setelah semua nilai dibaca. Tetapi saat itu nilai-nilai sebelumnya sudah hilang, karena setiap `cin` menimpa variabel yang sama.

Tampilan yang diinginkan

```
Nilai: 70 85 60 90 75 80
Rata-rata   : 76.67
Di atas rata: 3
```

TEBAK DULU **Coba pikirkan**

Apa yang perlu dilakukan supaya semua nilai masih ada saat rata-rata sudah diketahui? Berapa variabel yang dibutuhkan untuk 40 mahasiswa?

JAWABAN **Cara memecahnya**

Simpan semua: Semua nilai perlu disimpan, bukan hanya totalnya. Proses dua kali: Putaran pertama menghitung rata-rata, putaran kedua membandingkan. Terjemahkan: Satu array, dua `for` dengan indeks 0 sampai n − 1.

LANGKAH 1

Deklarasi dan indeks

Array adalah sederet variabel bertipe sama yang berbagi satu nama. Setiap elemen diakses dengan indeks di dalam kurung siku, dimulai dari 0. Array `int nilai[5]` punya elemen `nilai[0]` sampai `nilai[4]`.

Setiap elemen berperilaku seperti variabel biasa: bisa dibaca, ditugaskan, dan dipakai di ekspresi.

ANALOGI **Contoh sehari-hari**

Array seperti deretan loker bernomor di stasiun: satu nama deret, setiap loker punya nomor mulai dari 0, dan setiap loker berisi satu barang sejenis.

Bentuk umum

```
tipe nama[UKURAN] = {nilai, ...};
nama[indeks]           // indeks 0 .. UKURAN-1
int nilai[5] = {70, 85, 60, 90, 75};
```

array.cpp

```
int nilai[5] = {70, 85, 60, 90, 75};
cout << nilai[0] << " " << nilai[4] << endl;
nilai[2] = 65;
nilai[3] += 5;
for (int i = 0; i < 5; i++) {
    cout << nilai[i] << " ";
}
cout << endl;
```

TEBAK DULU

Sebelum menjalankan program, tulis keluarannya di kertas, karakter demi karakter.

JAWABAN

Keluaran

```
70 75
70 85 65 95 75
```

Array berukuran 5 punya indeks 0 sampai 4. Ukuran harus konstanta.

LANGKAH 2

Mengisi array dari masukan

Bila banyak data baru diketahui saat program berjalan, deklarasikan array dengan ukuran maksimum yang masuk akal, lalu simpan banyak data sebenarnya di variabel `n`. Semua perulangan memakai batas `i < n`.

ANALOGI **Contoh sehari-hari**

Seperti mengisi loker satu per satu dari nomor 0 sampai nomor terakhir yang dipakai.

Bentuk umum

```
int n, a[100];
cin >> n;
for (int i = 0; i < n; i++)
    cin >> a[i];
```

baca.cpp

```
int n, a[100];
cin >> n;
for (int i = 0; i < n; i++) {
    cin >> a[i];
}
int total = 0;
for (int i = 0; i < n; i++) {
    total += a[i];
}
cout << "Total " << total << endl;
```

Masukan

```
4
10 20 30 40
```

TEBAK DULU

Sebelum menjalankan program, tulis keluarannya di kertas, karakter demi karakter.

JAWABAN

Keluaran

```
Total 100
```

Deklarasikan array cukup besar (misalnya 100), lalu pakai hanya n elemen pertama.

LANGKAH 3

Mencari maksimum dan posisinya

Untuk mencari maksimum, anggap elemen pertama terbesar, lalu bandingkan dengan elemen berikutnya satu per satu. Menyimpan indeks lebih berguna daripada menyimpan nilai, karena indeks juga memberi tahu posisinya.

ANALOGI **Contoh sehari-hari**

Seperti juri lomba yang memegang juara sementara, lalu menggantinya setiap kali ada peserta yang lebih baik.

Bentuk umum

```
int iMaks = 0;
for (int i = 1; i < n; i++)
    if (a[i] > a[iMaks]) iMaks = i;
```

maks.cpp

```
int a[6] = {70, 85, 60, 90, 75, 80};
int iMaks = 0;
for (int i = 1; i < 6; i++) {
    if (a[i] > a[iMaks]) {
        iMaks = i;
    }
}
cout << a[iMaks] << " di indeks " << iMaks << endl;
```

TEBAK DULU

Sebelum menjalankan program, tulis keluarannya di kertas, karakter demi karakter.

JAWABAN

Keluaran

```
90 di indeks 3
```

Simpan indeks, bukan nilainya. Dari indeks, nilai selalu bisa diambil kembali.

LANGKAH 4

Dua putaran: rata-rata lalu bandingkan

Inilah jawaban masalah pembuka. Putaran pertama menghitung rata-rata. Putaran kedua membandingkan setiap nilai dengan rata-rata itu. Tanpa array, putaran kedua mustahil dilakukan karena nilai-nilainya sudah hilang.

ANALOGI **Contoh sehari-hari**

Untuk tahu siapa di atas rata-rata, kita harus menghitung rata-rata dulu, lalu memeriksa ulang setiap nilai. Kalau nilainya sudah dibuang, pemeriksaan ulang tidak mungkin.

diatas.cpp

```
int a[6] = {70, 85, 60, 90, 75, 80};
int total = 0;
```

```
for (int i = 0; i < 6; i++) total += a[i];
double rata = total / 6.0;
int diAtas = 0;
for (int i = 0; i < 6; i++) {
    if (a[i] > rata) diAtas++;
}
cout << rata << " " << diAtas << endl;
```

TEBAK DULU

Sebelum menjalankan program, tulis keluarannya di kertas, karakter demi karakter.

JAWABAN

Keluaran

```
76.6667 3
```

LANGKAH 5

Di luar batas

C++ tidak memeriksa apakah indeks berada di dalam array. Membaca `a[3]` dari array berukuran 3 tidak menghasilkan error kompilasi; program membaca memori di sebelah array. Hasilnya bisa acak, bisa sama di komputer kalian tetapi berbeda di komputer lain, atau program berhenti mendadak.

Kebiasaan yang mencegahnya: selalu tulis syarat perulangan sebagai `i < n`, bukan `i <= n`.

ANALOGI **Contoh sehari-hari**

Seperti membuka loker nomor 11 di deretan 10 loker: yang terbuka adalah milik orang lain. Program tidak memperingatkan, hasilnya bisa acak.

batas.cpp

```
int a[3] = {1, 2, 3};
int jumlah = 0;
for (int i = 0; i < 3; i++) {
    jumlah += a[i];
}
```

```
cout << jumlah << endl;
// for (int i = 0; i <= 3; i++) membaca a[3]:
// di luar array, hasilnya tidak bisa ditebak
```

TEBAK DULU

Sebelum menjalankan program, tulis keluarannya di kertas, karakter demi karakter.

JAWABAN

Keluaran

```
6
```

C++ tidak memeriksa batas indeks. Program bisa tetap jalan dengan hasil acak, atau berhenti mendadak.

LANGKAH 6

Menggeser dan membalik

Operasi yang memindahkan elemen, seperti rotasi, paling mudah dipahami dengan menelusuri isi array setelah setiap putaran. Perhatikan arah pergeseran: menggeser ke kanan harus dimulai dari belakang. Bila dimulai dari depan, nilai di depan menimpa nilai berikutnya sebelum sempat dipindah.

> ANALOGI **Contoh sehari-hari**
>
> Seperti menggeser antrean ke belakang satu langkah: orang paling belakang harus pindah lebih dulu, kalau tidak ia tertabrak orang di depannya.

Langkah	Isi array	Keterangan
awal	10 20 30 40 50	simpan = 50
i = 4	10 20 30 40 40	a[4] = a[3]
i = 3	10 20 30 30 40	a[3] = a[2]
i = 1	10 10 20 30 40	setelah i = 2 dan i = 1
akhir	50 10 20 30 40	a[0] = simpan

WASPADA

Pesan error yang sering muncul

Semua pesan di tabel ini dihasilkan g++ dengan opsi `-Wall`, sama seperti yang dipakai `cek.sh`.

Kesalahan	Contoh	Pesan pertama dari g++
Ukuran negatif	`int a[-3];`	`error: narrowing conversion of '-3' from 'int' to 'long unsigned int'`
Inisialisasi kelebihan elemen	`int a[2] = {1, 2, 3};`	`error: too many initializers for 'int [2]'`
Indeks bukan bilangan bulat	`a[1.5]`	`error: invalid types 'int [3][double]' for array subscript`
Menugaskan array ke array	`b = a;`	`error: invalid array assignment`
Ukuran tidak ditulis	`int a[];`	`error: storage size of 'a' isn't known`

Ukuran array dari variabel, misalnya `int a[n]` dengan `n` dibaca dari `cin`, diterima g++ tanpa pesan apa pun sebagai ekstensi, tetapi itu bukan C++ standar dan ditolak compiler lain. Di mata kuliah ini, ukuran array selalu konstanta.

Akses di luar batas, misalnya `a[5]` pada `int a[5]` atau `a[i]` dengan `i` = n, tidak dilaporkan saat kompilasi. Itulah sebabnya tracing indeks penting.

LATIHAN

Latihan mandiri

Latihan ini sama dengan hands-on di kelas. Semua file ada di `02 Hands-on/P9 - Array 1D - Hands-on/latihan/`. Kerjakan berurutan, lalu cocokkan dengan `expected/` atau jalankan `bash cek.sh`.

No	File	Tingkat	Yang dilatih
1	`handson1-nilai.cpp`	Dasar	mengisi dan menampilkan, urutan terbalik
2	`handson2-statistik.cpp`	Menengah	rata-rata, maksimum, dua putaran
3	`handson3-geser.cpp`	Tantangan	telusuri isi array, perbaiki tiga kesalahan indeks
+	`bonus-frekuensi.cpp`	Pengayaan	array sebagai pencacah

Latihan 1: handson1-nilai.cpp

Baca n nilai ke array, tampilkan dengan indeksnya, lalu dalam urutan terbalik.

Latihan 2: handson2-statistik.cpp

Hitung rata-rata, nilai tertinggi beserta indeksnya, dan banyak nilai di atas rata-rata.

Latihan 3: handson3-geser.cpp

Rotasi kanan yang salah indeks. Gambar isi array sesudah setiap putaran, perbaiki, dan jelaskan.

Bonus: bonus-frekuensi.cpp

Histogram frekuensi nilai per rentang dengan array pencacah.

Latihan menelusuri

tebak.cpp

```cpp
int a[5] = {3, 1, 4, 1, 5};
for (int i = 1; i < 5; i++) {
    a[i] = a[i] + a[i - 1];
}
for (int i = 0; i < 5; i++) {
    cout << a[i] << " ";
}
cout << endl;
```

TEBAK DULU

Tulis keluarannya di kertas tanpa menjalankan program. Buat trace table bila perlu.

JAWABAN

Keluaran

```
3 4 8 9 14
```

Setiap elemen ditambah elemen sebelumnya yang sudah diperbarui, sehingga terbentuk jumlah kumulatif: 3, 4, 8, 9, 14.

PENUTUP

Rangkuman

Konsep	Yang perlu diingat
Deklarasi dan indeks	Array berukuran 5 punya indeks 0 sampai 4.
Mengisi array dari masukan	Deklarasikan array cukup besar (misalnya 100), lalu pakai hanya n elemen pertama.
Mencari maksimum dan posisinya	Simpan indeks, bukan nilainya.
Dua putaran: rata-rata lalu bandingkan	Inilah jawaban masalah pembuka.
Di luar batas	C++ tidak memeriksa batas indeks.
Menggeser dan membalik	Operasi yang memindahkan elemen, seperti rotasi, paling mudah dipahami dengan menelusuri isi array setelah setiap putaran.

PENUTUP

Cek pemahaman

Jawab tanpa menjalankan program, lalu periksa dengan menjalankannya.

1. Apa indeks elemen terakhir dari `double d[12]`?
2. Mengapa maksimum sebaiknya disimpan sebagai indeks?
3. Apa yang terjadi bila program membaca `a[10]` dari `int a[10]`?
4. Mengapa menghitung banyak nilai di atas rata-rata butuh array?
5. Rotasi kanan harus menggeser dari arah mana, dan mengapa?

> JAWABAN
>
> 1. 11.
> 2. Indeks memberi nilai sekaligus posisinya.

3. Perilaku tidak terdefinisi; hasil acak atau program berhenti.
4. Rata-rata baru diketahui setelah semua nilai dibaca, jadi nilai harus disimpan untuk dibandingkan.
5. Dari belakang, agar tidak menimpa nilai yang belum dipindah.

PENILAIAN

Exit Ticket 9

Dikerjakan di 25 menit terakhir kelas, sendiri, tanpa AI, internet, atau diskusi. Exit Ticket 9 adalah satu dari 14 Exit Ticket; rata-ratanya menjadi komponen berbobot 25%.

Soal	Isi	Poin	CPMK
A1	Tulis program yang membaca n nilai ke array dan menampilkan nilai minimum beserta indeksnya	25	CPMK0607
A2	Tulis program yang menghitung banyak elemen genap dan ganjil	25	CPMK0607
B1	Telusuri isi array setelah sebuah perulangan yang mengubah elemennya	20	CPMK0608
B2	Jelaskan mengapa sebuah perulangan menyebabkan akses di luar batas	20	CPMK0608
B3	Jelaskan mengapa sebuah masalah butuh array, bukan variabel tunggal	10	CPMK0608

PENILAIAN

Tugas 9: Sistem Nilai Mahasiswa

Kumpulkan sebelum Pertemuan 10 lewat kanal pengumpulan yang diumumkan dosen.

Bagian A: program (50 poin)

Buat program dengan ketentuan berikut. Gunakan `const` untuk ukuran array.

1. Baca jumlah mahasiswa (maksimal 50)
2. Baca NIM dan nilai (0 sampai 100) setiap mahasiswa ke dua array sejajar, dengan validasi
3. Tampilkan semua data, nilai tertinggi dan terendah, serta rata-rata kelas
4. Tampilkan jumlah LULUS (nilai 60 ke atas), TIDAK LULUS, dan persentase kelulusan
5. Fitur pencarian: baca sebuah NIM, tampilkan nilainya atau pesan tidak ditemukan

> ANALOGI **Bonus, tidak wajib**
>
> Urutkan nilai dari tertinggi ke terendah (materi sorting di Pertemuan 12).

Bagian B: penjelasan (50 poin)

Komentar maksimal 150 kata dan gambar isi kedua array setelah membaca tiga data contoh. Jelaskan cara menjaga indeks tetap di dalam batas, dan mengapa dua array harus diproses dengan indeks yang sama.

Rubrik Tugas 9

Kriteria	Poin	Sangat baik 85–100	Baik 70–84	Cukup 55–69	Kurang <55
A1 Program berjalan	20	tanpa error dan peringatan	ada peringatan	perlu satu perbaikan	tidak terkompilasi
A2 Kelengkapan fitur	15	semua statistik dan pencarian	satu fitur kurang	dua kurang	tiga atau lebih kurang
A3 Validasi dan ketepatan	15	validasi tepat, hasil benar untuk n = 1 dan n = 10	satu salah	dua salah	sebagian besar salah
B1 Isi array dan batas indeks	30	gambar tepat, alasan jelas	satu langkah keliru	gambar tanpa alasan	tidak ada atau disalin
B2 Cerita error dan pengujian	20	pesan, sebab, perbaikan, uji	pesan tidak dikutip	hanya perbaikan	tidak ada

Nilai kriteria = poin × skor level. Contoh: A1 di level Baik dengan skor 80 memberi 20 × 0,80 = 16 poin.

PENUTUP

Bacaan lanjut dan persiapan minggu depan

1. Deitel, P. dan Deitel, H. C++ How to Program, edisi ke-10, Bab 7.
2. learncpp.com, Bab 17 (C-style arrays).
3. Gaddis, T. Starting Out with C++, Bab 7.
4. Slide `01 Slide/P9 Array Satu Dimensi.pdf` dan hands-on di `02 Hands-on/P9 - Array 1D - Hands-on/`.

> PENTING **Minggu depan**
>
> Pertemuan 10 membahas array dua dimensi dan string, sejalan dengan Algoritma Pertemuan 10.